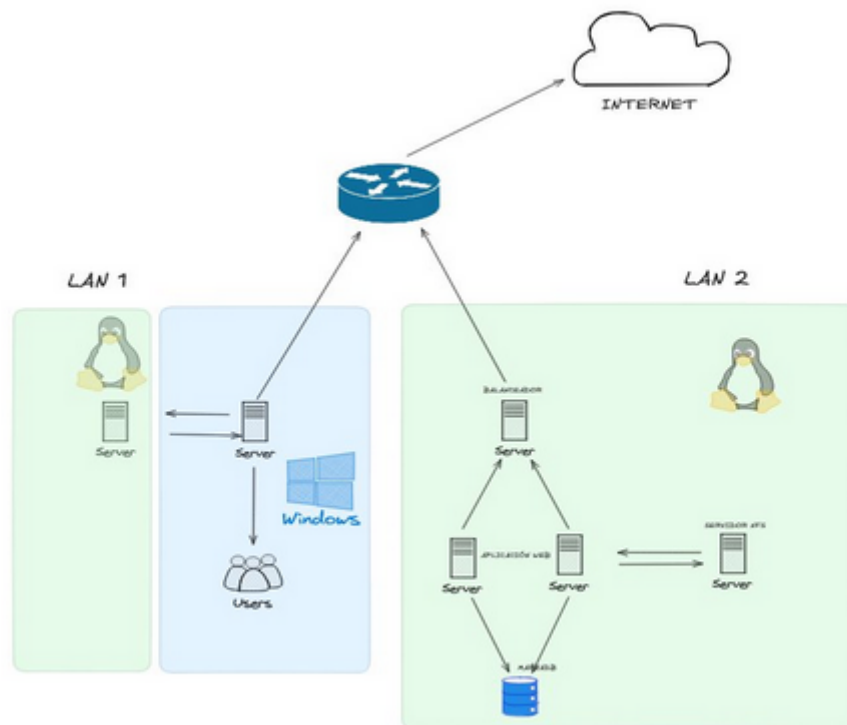


PROYECTO INTERMODULAR



ANTONIO CUEVAS FERNÁNDEZ

15/06/2026

ASIR 2

1.- ÍNDICE

1.- ÍNDICE	2
2.- INTRODUCCIÓN	5
2.1.- ALCANCE DE LA INFRAESTRUCTURA	5
2.2.- PÚBLICO DESTINATARIO	5
3.- DESCRIPCIÓN GENERAL DE LA ARQUITECTURA	6
4.- REQUISITOS DEL SISTEMA	7
4.1.- REQUISITOS HARDWARE Y DE SISTEMA OPERATIVO	7
4.2.- REQUISITOS SOFTWARE	8
5.- DISEÑO DE LA RED	9
5.1.- ESTRUCTURA DE LA RED	9
5.2.- PROTOCOLOS Y PUERTOS	9
5.3.- FIREWALL	10
6.- SERVIDOR WEB	11
6.1.- FUNCIONAMIENTO DENTRO DE LA INFRAESTRUCTURA	11
6.2.- CONFIGURACIÓN BÁSICA	12
6.3.- VIRTUALHOST/ SITIOS WEB	12
6.4.- GESTIÓN DE CERTIFICADO HTTPS	13
6.5.- UBICACIÓN DE LOS ARCHIVOS	13
6.6.- LOGS Y REGISTROS	14
7.- BALANCEADOR DE CARGA	14
8.- BASE DE DATOS	15
8.1.- MOTOR DE BASE DE DATOS	15
8.2.- ARQUITECTURA	15
8.3.- CONFIGURACIÓN BÁSICA	15
8.4.- USUARIOS Y PERMISOS	15
8.5.- CONEXIÓN SEGURA CON LOS SERVIDORES WEB	16
9.- INFRAESTRUCTURA DOCKER (LAN 2)	16
9.1.- ARQUITECTURA DE CONTENEDORES	16
9.2.- DESCRIPCIÓN DEL docker-compose.yml	17
9.3.- REDES DOCKER	17

10.- SERVICIOS CORPORATIVOS EN LAN 1 (DOCKER)	18
10.1.- GLPI — GESTIÓN DE ACTIVOS Y SOPORTE	18
10.2.- PASSBOLT — GESTOR DE CONTRASEÑAS	18
10.3.- PROXY INVERSO	19
10.4.- SERVICIO DNS (Domain Name System)	20
10.5.- SERVICIO DHCP (Dynamic Host Configuration Protocol)	20
10.6.- SERVICIO FTP (File Transfer Protocol)	21
11.- SISTEMA DE RESPALDO	21
11.1.- OBJETIVOS DEL SISTEMA RESPALDADO	21
11.2.- ELEMENTOS RESPALDADOS	21
11.3.- TIPOS DE COPIAS Y FRECUENCIA	22
11.4.- UBICACIÓN DE LAS COPIAS	22
11.5.- PROCEDIMIENTO Y RESTAURACIÓN	22
12.- ALTA DISPONIBILIDAD Y TOLERANCIA A FALLOS	23
12.1.- ANÁLISIS DE PUNTOS CRÍTICOS (SPOF)	23
12.2.- MECANISMOS DE REDUNDANCIA Y RESILIENCIA	23
12.3.- ESCENARIO DE FALLOS Y RESPUESTA DEL SISTEMA	24
12.4.- RECUPERACIÓN ANTE DESASTRES	24
13.- SEGURIDAD	25
13.1.- GESTIÓN DE USUARIOS Y ACCESO	25
13.2.- POLÍTICAS DE CONTRASEÑAS Y SECRETOS	25
13.3.- ACTUALIZACIONES Y PARCHES DE SEGURIDAD	26
13.4.- PROTECCIÓN FRENTE A ATAQUES	26
13.5.- BUENAS PRÁCTICAS DE SEGURIDAD	26
14.- MONITORIZACIÓN Y MANTENIMIENTO	27
14.1.- HERRAMIENTA DE MONITORIZACIÓN	27
14.2.- PARÁMETROS SUPERVISADOS	27
14.3.- GESTIÓN DE ALERTAS	28
14.4.- TAREAS DE MANTENIMIENTO PERIÓDICO	28
15.- PROCEDIMIENTOS OPERATIVOS	28
15.1.- ARRANQUE Y PARADAS DE SERVICIOS	28
15.2.- ALTA Y BAJA DE UN SERVIDOR WEB	29

15.3.- ACTUALIZACIÓN DEL SISTEMA	29
15.4.- RESTAURACIÓN DE BACKUPS	29
16.- TROUBLESHOOTING	30
16.1.- PROBLEMAS HABITUALES	30
16.2.- DIAGNÓSTICOS BÁSICOS Y COMPROBACIONES	30
16.3.- LOGS RELEVANTES	31

2.- INTRODUCCIÓN

El presente documento detalla el diseño, despliegue y gestión de la infraestructura informática centralizada en las oficinas principales de la organización. Este entorno ha sido proyectado para garantizar un ecosistema digital robusto, seguro y eficiente, que actúe como el núcleo de todas las operaciones diarias de la empresa.

2.1.- ALCANCE DE LA INFRAESTRUCTURA

La infraestructura descrita se encuentra estrictamente delimitada al ámbito físico del edificio corporativo. El alcance técnico comprende desde la capa física (cableado estructurado y salas de telecomunicaciones) hasta la capa lógica (contenedores Docker, segmentación de redes y servicios de conectividad).

Al tratarse de una infraestructura confinada a una ubicación física única, se prioriza la baja latencia y la alta disponibilidad de los recursos internos, asegurando que el flujo de datos entre departamentos sea inmediato y fluido, sin dependencia directa de servicios externos para la operatividad básica de la red de área local (LAN).

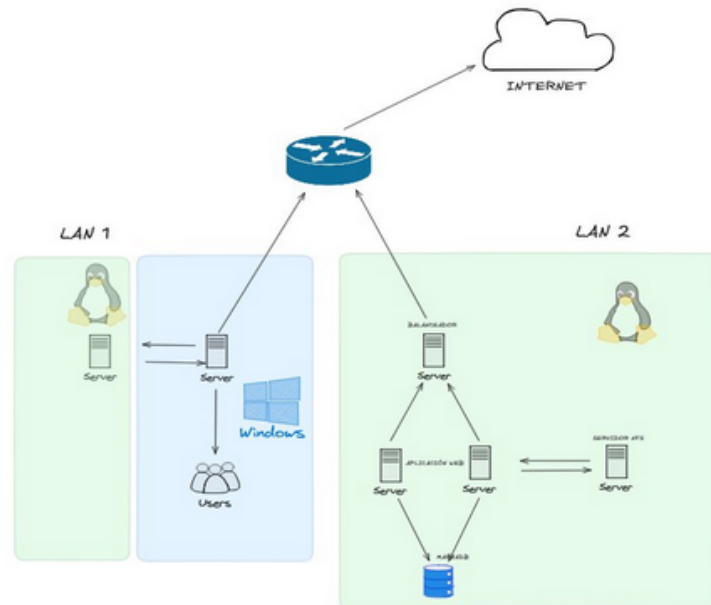
2.2.- PÚBLICO DESTINATARIO

- **Administradores de Sistemas y Redes:** Responsables de la configuración, mantenimiento preventivo y resolución de incidencias de nivel avanzado.
- **Técnicos de Soporte:** Personal encargado de la asistencia directa a los terminales de usuario y la gestión de periféricos dentro del edificio.
- **Dirección de IT:** Para la supervisión estratégica y la toma de decisiones sobre futuras escalabilidades o auditorías de seguridad.
- **Usuarios (Empleados de la empresa):** todos los activos de la empresa que tengan acceso a los servicios de los servidores.

3.- DESCRIPCIÓN GENERAL DE LA ARQUITECTURA

La Arquitectura de la red consistirá en dividir la red en 2 partes, una LAN la cual se conectará un servidor de Windows Server y la segunda LAN donde tendremos una red la cual será hecha con Linux Debian para la infraestructura de nuestra página WEB. Ambas LANs estarán conectadas entre sí mediante un router y este a su vez estará conectado a Internet.

- **LAN Windows Server:** En esta LAN el servidor de Windows Server el cual dará servicio de DNS, DHCP y FTP a los trabajadores que se encuentren dentro de la empresa para que puedan subir o bajar los archivos del servidor y tener conexión a internet. Para la gestión de usuarios de Active Directory se utiliza una aplicación Python con interfaz gráfica (Tkinter) que ejecuta scripts PowerShell (crear_usuario.ps1, eliminar_usuario.ps1, listar_usuarios.ps1) de forma automatizada. Adicionalmente, sobre la misma máquina host Linux de LAN 1 se despliegan mediante Docker los servicios corporativos internos: GLPI (gestión de activos), Passbolt (gestión de contraseñas corporativas) y un Proxy Inverso para centralizar su acceso.
- **LAN Linux:** En esta LAN se monta toda la infraestructura de la aplicación web mediante Docker: el balanceador de carga Nginx, 2 servidores backend PHP-FPM donde se aloja la aplicación web, y un servidor de base de datos MySQL, el cual almacenará los datos de la aplicación web. Esta LAN tiene acceso a Internet tanto entrante (puertos 80 y 443) como saliente, permitiendo por ejemplo la renovación de certificados SSL y actualizaciones.



4.- REQUISITOS DEL SISTEMA

4.1.- REQUISITOS HARDWARE Y DE SISTEMA OPERATIVO

WINDOWS SERVER

TIPO DE SISTEMA	MEMORIA RAM	CPU (MÍNIMOS)	VERSIÓN SO
Cliente	2 GB (Mínimo)4 GB (Recomendado)	2 núcleosArquitectura amd64	Windows 10 Pro
Servidor	2 GB (Mínimo)4 GB (Recomendado)	2 núcleosArquitectura amd64	Windows Server 2022

LINUX DEBIAN 13

SERVIDOR	MEMORIA RAM	CPU	VERSIÓN SO
Balanceador / Nginx	512 MB (Mínimo)	1 NúcleoArquitectura amd64	Debian 13
Backend PHP-FPM	512 MB (Mínimo)	1 NúcleoArquitectura amd64	Debian 13
Base de Datos	512 MB (Mínimo)	1 NúcleoArquitectura amd64	Debian 13
Host LAN 1 (GLPI, Passbolt, Proxy)	2 GB (Mínimo)	2 NúcleosArquitectura amd64	Debian 13

4.2.- REQUISITOS SOFTWARE

SERVICIO	SISTEMA OPERATIVO	SOFTWARE USADO	VERSIONES
Balanceador / Web	Debian 13	Docker + Nginx	Docker 26.x · Nginx latest
Backend	Debian 13	Docker + PHP-FPM	PHP 8.2-fpm-alpine
Base de datos	Debian 13	Docker + MySQL	MySQL 8.0
GLPI	Debian 13	Docker + GLPI	GLPI 11.0.6
Passbolt	Debian 13	Docker + Passbolt CE	Passbolt CE latest
Proxy Inverso LAN 1	Debian 13	Docker + Nginx	Nginx latest
Windows Server	Windows Server 2022	DNS, DHCP, FTP	Roles nativos WS 2022

5.- DISEÑO DE LA RED

5.1.- ESTRUCTURA DE LA RED

El diseño de la red que se presenta aquí es de tipo lógica, debido a que no se presentan ningún otro componente para la estructura de la red (switches, impresoras, etc).

Zona	Propósito	Rango de Red
LAN 1 (Windows + Docker)	Usuarios internos + servicios corporativos (Solo salida a Internet)	192.168.2.0/24
LAN 2 (DMZ)	Balanceador y Servidores Web (Acceso entrante puertos 80/443 + salida a Internet)	192.168.3.0/24
red-pública (Docker)	Nginx, backends PHP (red bridge interna Docker LAN 2)	Interna Docker
red-privada (Docker)	Backends PHP ↔ MySQL (totalmente aislada)	Interna Docker
Red de Backend	Base de Datos MySQL (MariaDB)	192.168.3.0/24

5.2.- PROTOCOLOS Y PUERTOS

- **Balanceador:** Los protocolos que van a seguir el servicio de balanceador es el protocolo de HTTP (Hypertext Transfer Protocol) y el protocolo HTTPS (Hypertext Transfer Protocol Secure) usando el puerto 80 y el puerto 443 respectivamente.
- **Backends PHP-FPM:** Se comunican con el balanceador por la red-publica de Docker (puerto 9000 interno). Se comunican con MySQL a través de la red-privada de Docker (puerto 3306 interno, no expuesto al host).
- **MariaDB / MySQL:** Los servidores web tendrán que realizar las consultas de las bases de datos a través del puerto 3306 del servidor donde se aloja

MySQL. Este puerto está estrictamente confinado a la red-privada de Docker y no tiene mapeo al host.

- **GLPI (LAN 1):** El stack GLPI expone un Nginx interno en los puertos 8081 (HTTP) y 40443 (HTTPS) del host. El proxy inverso externo gestiona el acceso desde glpi.aromaris.local.
- **Passbolt (LAN 1):** Accesible en el host de LAN 1 por los puertos 8080 (HTTP) y 4443 (HTTPS).
- **Proxy Inverso (LAN 1):** Escucha en los puertos 80 (HTTP) y 443 (HTTPS) del host de LAN 1, enrutando hacia GLPI (glpi.aromaris.local) y Passbolt (passbolt.aromaris.local) según el Virtual Host mediante las redes Docker externas.

5.3.- FIREWALL

WINDOWS

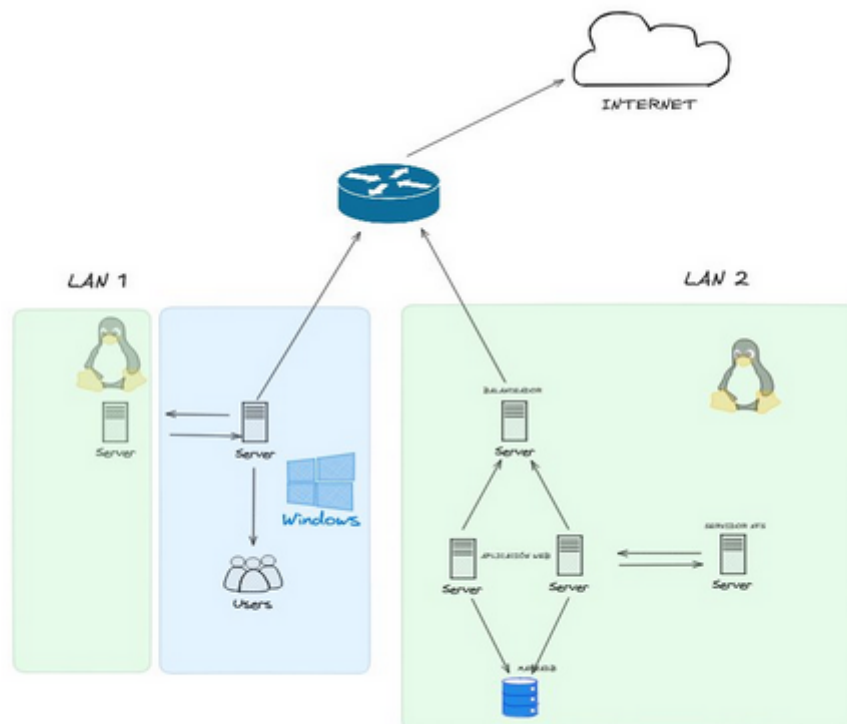
En la parte de Windows Server, el firewall tiene que permitir el acceso a internet, pero que internet no tenga acceso al servidor. También a través del router, el servidor windows tiene que tener acceso a los servidores de LAN 2 para administrar y consultar la base de datos.

LINUX

- **LAN 2 — Acceso exterior controlado:** La LAN 2 es accesible desde Internet por los puertos TCP 80 y 443 destinados al host Docker que ejecuta el balanceador Nginx. Adicionalmente, LAN 2 tiene acceso saliente a Internet mediante NAT/MASQUERADE para tareas como renovación de certificados SSL y actualizaciones de imágenes Docker. Cualquier otro puerto de entrada desde Internet es rechazado por el firewall.
- **LAN 1 — Solo salida:** LAN 1 tiene acceso a Internet mediante NAT/MASQUERADE, pero Internet no puede iniciar conexiones hacia LAN 1. Los servicios Docker internos (GLPI, Passbolt, proxy inverso) son accesibles únicamente desde dentro de LAN 1.
- El balanceador tiene que redirigir el tráfico que llega a los puertos 80 y 443 de los protocolos HTTP y HTTPS de la IP pública a la IP privada (Port Forwarding).

- El servidor de MySQL y el servidor de base de datos no deben tener "Default Gateway" hacia el exterior o deben tener una regla de firewall que prohíba estrictamente cualquier tráfico que no provenga de los Servidores Web.

El script enrutamiento.sh implementa estas reglas mediante iptables con política por defecto FORWARD DROP, permitiendo explícitamente el tráfico saliente de ambas LANs y el entrante a LAN 2 por los puertos 80 y 443.



6.- SERVIDOR WEB

6.1.- FUNCIONAMIENTO DENTRO DE LA INFRAESTRUCTURA

El servidor web actúa como el motor de procesamiento de la arquitectura. Su función principal es recibir las peticiones procesadas por el balanceador de carga, ejecutar la lógica de negocio, interactuando con la base de datos si es necesario, y devolver el contenido dinámico o estático al usuario final. La aplicación web alojada se denomina Aromaris, una tienda de jabones artesanales que incluye un sistema de autenticación de empleados con sesiones PHP y consultas preparadas contra la base de datos Usuarios.

Se ha implementado Nginx en sustitución de Apache, debido a su modelo de concurrencia basado en eventos (event-driven, non-blocking I/O), que ofrece un rendimiento superior en escenarios con alto número de conexiones simultáneas y un consumo de memoria significativamente menor. En la arquitectura Docker, el contenedor nginx-balanceador actúa simultáneamente como terminador SSL/TLS, servidor de archivos estáticos y proxy inverso hacia los backends PHP-FPM.

6.2.- CONFIGURACIÓN BÁSICA

Se ha implementado Nginx debido a su alto rendimiento en el manejo de conexiones simultáneas y su bajo consumo de recursos.

- **Procesos:** Configurado para ajustar automáticamente el número de procesos workers según la cantidad de núcleos de CPU disponibles (worker_processes auto).
- **Timeouts:** Ajuste de keepalive_timeout, proxy_read_timeout y proxy_connect_timeout para evitar que procesos lentos bloqueen el servidor, garantizando la fluidez del servicio.
- **Archivos estáticos:** Nginx sirve directamente los archivos estáticos (CSS, JS, imágenes) montados desde el volumen ./html, sin pasar por PHP-FPM, maximizando el rendimiento.

6.3.- VIRTUALHOST/ SITIOS WEB

El servidor está configurado mediante bloques `server{}` en `nginx.conf`, lo que permite una gestión modular del sitio:

- **Aislamiento:** Cada entorno tiene su propio archivo de configuración independiente.
- **Nombres de dominio:** El servidor responde exclusivamente a las peticiones dirigidas al dominio oficial `aromaris.org` y `www.aromaris.org` (`server_name`), rechazando peticiones directas por dirección IP para mejorar la seguridad.
- **Upstream backends:** Se define un bloque `upstream php_backends` con los dos contenedores backend PHP-FPM (`backend1:9000` y `backend2:9000`), aprovechando la resolución DNS interna de Docker dentro de la red-privada.

6.4.- GESTIÓN DE CERTIFICADO HTTPS

La seguridad en el transporte de datos es obligatoria. El servidor web gestiona el cifrado de extremo a extremo:

- **Certificados:** Se utilizan certificados SSL/TLS autofirmados para garantizar el cifrado de las comunicaciones. Estos certificados se generan localmente, se almacenan en el volumen `./certs` y se montan en el contenedor de Nginx en la ruta `/etc/nginx/certs`. El script `certificados.sh` supervisa la caducidad de todos los certificados del sistema y envía alertas por correo electrónico cuando quedan menos de 30 días para su expiración, permitiendo así su regeneración manual antes de que caduquen.
- **Redirección Forzosa:** Se ha configurado una regla en Nginx que redirige automáticamente todo el tráfico entrante por el puerto 80 (HTTP) hacia el puerto 443 (HTTPS).
- **Cifrado Fuerte:** Solo se permiten protocolos modernos (TLS 1.2 o superior), desactivando versiones vulnerables como SSLv3 o TLS 1.0.

6.5.- UBICACIÓN DE LOS ARCHIVOS

Para mantener una estructura limpia y fácil de mantener, los archivos del proyecto se organizan de la siguiente manera:

- **Raíz del sitio:** El volumen `./html` se monta en `/var/www/html` tanto en el contenedor Nginx como en los backends PHP, garantizando consistencia entre todos los nodos.
- **Configuración Nginx:** El archivo `./nginx/nginx.conf` se monta en modo solo lectura (`:ro`) en `/etc/nginx/nginx.conf` para evitar modificaciones accidentales en tiempo de ejecución.
- **Permisos:** El propietario de los archivos es un usuario de sistema sin privilegios de shell, limitando el alcance de una posible vulneración.

6.6.- LOGS Y REGISTROS

Los archivos de registro (logs) en un servidor web son registros automáticos de texto que documentan cada actividad y evento, como solicitudes de páginas, direcciones IP de usuarios, errores y tiempos de acceso, permitiendo analizar el comportamiento del visitante, optimizar el rendimiento y diagnosticar problemas de seguridad o técnicos.

- `access.log`: registra cada petición HTTP con código de respuesta, IP de origen y tiempo de respuesta.
- `error.log`: registra errores de Nginx, problemas de conexión con los backends y fallos de configuración.
- Los logs pueden consultarse con: `docker logs nginx-balanceador`

7.- BALANCEADOR DE CARGA

El balanceador de cargas es el contenedor `nginx-balanceador`, que distribuye las cargas de red a los distintos servidores backend. En la arquitectura Docker, Nginx actúa nativamente como balanceador de carga sin necesidad de un componente externo adicional.

El tipo de balanceo que se realizará en esta red es de tipo Round Robin, el balanceador distribuirá la red de forma secuencial a los servidores, es decir, que cada ciertas solicitudes se redirigirá a un servidor u a otro.

- **CONFIGURACIÓN DE SERVIDORES BACKEND:** La configuración consistirá en un nodo de 2 servidores backend (backend1 y backend2), referenciados por su nombre de servicio Docker dentro de la red-privada.
- **COMPROBACIÓN DEL ESTADO DEL SERVICIO:** La comprobación del servicio consistirá en un monitoreo constante de los servidores, verificando su ruta de acceso. Nginx detecta automáticamente si un backend no responde y deja de enviarle tráfico hasta que se recupere (passive health checks).
- **ESCALABILIDAD:** Añadir un tercer backend es tan sencillo como añadir un nuevo servicio en docker-compose.yml e incluirlo en el bloque upstream de nginx.conf, sin tiempo de inactividad.

8.- BASE DE DATOS

8.1.- MOTOR DE BASE DE DATOS

En la base de datos de la red, he optado por una configuración de un solo servidor utilizando MySQL 8.0 (imagen oficial mysql:8.0) desplegado como contenedor Docker, debido a su soporte para integridad referencial, su capacidad para manejar transacciones complejas (ACID) y su amplia documentación para la optimización de consultas. Se ha configurado con mysql_native_password como plugin de autenticación por defecto para garantizar compatibilidad con el driver PDO de PHP.

8.2.- ARQUITECTURA

La arquitectura que se usará en esta aplicación web será única, de un solo servidor de base de datos, ya que optó por la simplicidad y la reducción del costo para montar la infraestructura. El contenedor db-mysql no expone ningún puerto al host ni a redes externas, estando confinado exclusivamente a la red-privada de Docker.

8.3.- CONFIGURACIÓN BÁSICA

La configuración del servidor se ha ajustado para maximizar el aprovechamiento de los recursos de hardware disponibles:

- **Credenciales:** Gestionadas mediante variables de entorno en docker-compose.yml (MYSQL_ROOT_PASSWORD, MYSQL_DATABASE: Usuario, MYSQL_USER: antonio, MYSQL_PASSWORD). En producción deben externalizar mediante Docker Secrets.
- **Persistencia:** Los datos se almacenan en el volumen Docker datos-mysql, que sobrevive a la recreación del contenedor.
- **Codificación:** Uso de UTF-8 para garantizar la compatibilidad internacional de caracteres.

8.4.- USUARIOS Y PERMISOS

Se aplica el Principio de Menor Privilegio (PoLP) para la administración de accesos:

- **Usuario Administrador:** Acceso restringido únicamente para tareas de mantenimiento y cambios de esquema.
- **Usuario de Aplicación:** Se ha creado un usuario específico (antonio) para el servidor web con permisos limitados a sentencias SELECT, INSERT, UPDATE y DELETE sobre las tablas necesarias, sin permisos de creación o eliminación de objetos (DROP, ALTER).
- **Autenticación:** Uso de algoritmos de cifrado fuerte para contraseñas.

8.5.- CONEXIÓN SEGURA CON LOS SERVIDORES WEB

La comunicación entre la capa de aplicación y la base de datos se realiza bajo estrictos protocolos de seguridad:

- **Aislamiento de Red:** La base de datos se escucha únicamente en la red-privada de Docker, permaneciendo inaccesible desde el internet público. Los backends acceden a ella mediante el nombre de contenedor db-mysql.
- **Restricción por nombre de contenedor:** La configuración de MySQL puede restringir las conexiones a los hosts backend1 y backend2 únicamente.
- **Cifrado TLS/SSL:** Se recomienda forzar el uso de conexiones cifradas entre PHP y MySQL para evitar la interceptación de datos en tránsito.

9.- INFRAESTRUCTURA DOCKER (LAN 2)

9.1.- ARQUITECTURA DE CONTENEDORES

Toda la infraestructura de la LAN 2 (aplicación web) se despliega mediante un único archivo docker-compose.yml. Esto permite reproducir el entorno completo con un solo comando (docker compose up -d) y garantiza que todos los contenedores usen las mismas versiones de imágenes y la misma configuración de red.

9.2.- DESCRIPCIÓN DEL docker-compose.yml

CONTENEDOR	IMAGEN	PUERTOS (host)	REDES	FUNCIÓN
nginx-balanceador	nginx:latest	80:80, 443:443	red-pública	Balanceador de carga + terminador SSL
backend1	php:8.2-fpm-alpine	—	red-pública, red-privada	Backend PHP-FPM con extensión pdo_mysql
backend2	php:8.2-fpm-alpine	—	red-pública, red-privada	Backend PHP-FPM con extensión pdo_mysql
db-mysql	mysql:8.0	—	red-privada	Base de datos MySQL totalmente aislada

El volumen ./html es compartido entre nginx-balanceador y los backends PHP para servir la aplicación web Aromaris. El volumen ./datos-mysql persiste los datos de la base de datos MySQL independientemente del ciclo de vida del contenedor. Los certificados SSL se gestionan en el volumen ./certs.

9.3.- REDES DOCKER

- **red-pública (bridge):** Conecta el balanceador Nginx y los dos backends PHP. Solo el balanceador expone puertos al host (80 y 443). Los backends no son accesibles directamente desde el exterior.
- **red-privada (bridge):** Conecta exclusivamente los backends PHP con MySQL. La base de datos no tiene ningún puerto mapeado al host ni está en la red-pública. Esto implementa el aislamiento de red a nivel de contenedor, replicando el concepto de DMZ a nivel de software.

10.- SERVICIOS CORPORATIVOS EN LAN 1 (DOCKER)

Sobre el mismo host Linux Debian 13 de LAN 1, se despliegan mediante Docker tres servicios adicionales para los usuarios internos de la empresa: GLPI, Passbolt y un Proxy Inverso. Estos servicios están confinados a LAN 1 y no son accesibles desde Internet.

10.1.- GLPI — GESTIÓN DE ACTIVOS Y SOPORTE

- **Función:** Sistema de gestión de activos de TI (inventario de hardware/software), helpdesk y gestión de incidencias para los empleados y el equipo de soporte. Integra un chatbot de IA basado en Ollama (modelo gemma3:12b o similar) que permite a los usuarios consultar tickets e incidencias mediante lenguaje natural, a través de la API REST de GLPI.
- **Puerto en el host:** 8081 (HTTP) / 40443 (HTTPS). Nginx interno del stack GLPI.
- **Acceso:** Exclusivamente desde LAN 1. El firewall bloquea cualquier acceso a este puerto desde el exterior.
- **Datos:** Persistidos en volumen Docker. Utiliza su propia base de datos en un contenedor separado, también confinada a la red interna de Docker de LAN 1.

10.2.- PASSBOLT — GESTOR DE CONTRASEÑAS

- **Función:** Gestor de contraseñas corporativo open source, con soporte para equipos, permisos granulares y auditoría de accesos.
- **Puertos en el host:** 8080 (HTTP) y 4443 (HTTPS).
- **Acceso:** Exclusivamente desde LAN 1. Passbolt requiere acceso HTTPS para funcionar correctamente; el proxy inverso gestiona la terminación SSL con un certificado interno para la LAN.
- **Seguridad:** Passbolt cifra las contraseñas en el cliente con OpenPGP antes de enviarlas al servidor, garantizando que ni el servidor ni el administrador puedan leer las contraseñas en texto claro.

10.3.- PROXY INVERSO

- **Función:** Punto único de entrada para los servicios Docker de LAN 1. Enruta las peticiones hacia GLPI o Passbolt según el Virtual Host (nombre de dominio o ruta) de la petición.
- **Puerto en el host:** 80 (HTTP) y 443 (HTTPS). Punto de entrada principal para usuarios LAN 1.
- **Implementación:** Basado en Nginx (contenedor nginx-proxy), con bloques server separados para cada servicio (glpi.aromaris.local, passbolt.aromaris.local) y soporte para SSL con certificados autofirmados almacenados en ./certs. Se conecta a las redes externas glpi_default y passbolt_default para enrutar el tráfico.
- **Ventajas:** Centraliza la gestión de certificados internos y simplifica el acceso de los usuarios: acceden a un único dominio/puerto sin necesidad de recordar puertos distintos para cada servicio.

SERVICIO	PUERTO EN EL HOST	PROTOCOLO	NOTAS

GLPI	8081	HTTP	Nginx interno del stack GLPI
Passbolt	8080	HTTP	Acceso directo o vía proxy inverso
Passbolt	4443	HTTPS	Acceso directo o vía proxy inverso
Proxy Inverso	443	HTTPS	Punto de entrada principal para usuarios LAN 1

10.4.- SERVICIO DNS

- **Función dentro de la infraestructura:** Actúa como el resolutor de nombres interno para la LAN 1. Su función principal es traducir las peticiones de nombres de dominio locales (como glpi.aromaris.local o passbolt.aromaris.local) en las direcciones IP privadas correspondientes del host contenedor.
- **Implementación y Software:** Se utiliza el rol nativo DNS Server de Windows Server 2022, integrado con la base de datos de Active Directory para asegurar la actualización dinámica de los registros de los clientes de la red.
- **Configuración:**
 - **Zona de Búsqueda Directa:** aromaris.local (Registros tipo A para el servidor proxy inverso y los servicios corporativos).
 - **Zona de Búsqueda Inversa:** Subred 192.168.2.0/24 (Registros PTR para mapear IPs a nombres).

10.5.- SERVICIO DHCP

- **Función dentro de la infraestructura:** Automatiza y centraliza la asignación de configuraciones IP a todos los dispositivos de los usuarios internos conectados a la LAN 1. Esto evita conflictos de direcciones IP duplicadas y simplifica la administración de la red corporativa.
- **Configuración:**

- **Ámbito de Red:** Rango asignable desde 192.168.2.15 hasta 192.168.2.240 con la máscara 255.255.255.
- **Opciones de Ámbito:**
 - Puerta de enlace (Router): IP interna del router en la LAN 1 (192.168.2.1).
 - Servidor DNS: La propia IP estática del Windows Server (192.168.2.11).
- **Exclusiones/Reservas:** Direcciones IP fijas reservadas para servidores locales e infraestructura crítica (como el host Linux de servicios corporativos).

10.6.- SERVICIO FTP

- **Función dentro de la infraestructura:** Proporciona un espacio de almacenamiento de archivos centralizado y seguro para que los empleados de la organización puedan subir, bajar o compartir documentación corporativa de manera interna.
- **Implementación y Software:** Servicio FTP Server nativo, administrado mediante el rol de IIS (Internet Information Services) en Windows Server 2022.

11.- SISTEMA DE RESPALDO

11.1.- OBJETIVOS DEL SISTEMA RESPALDADO

El objetivo principal es garantizar la recuperación ante desastres y la integridad de la información. Se busca minimizar el RPO (Recovery Point Objective), asegurando que, en caso de fallo crítico en el nodo único, la pérdida de datos sea mínima y el tiempo de restauración sea lo más breve posible.

11.2.- ELEMENTOS RESPALDADOS

Para una restauración completa del servicio, se respaldan tres pilares fundamentales:

- **Base de Datos:** Volcado completo de la estructura y los datos (mediante mysqldump). Es el elemento más crítico.
- **Contenidos Web:** Archivos estáticos, multimedia y código fuente ubicados en el directorio ./html (volumen Docker).
- **Configuraciones del Sistema:** Archivos docker-compose.yml, nginx/nginx.conf, scripts de enrutamiento (enrutamiento.sh) y archivos .env de la aplicación.
- **Datos de servicios LAN 1:** Volúmenes Docker de GLPI y Passbolt.

11.3.- TIPOS DE COPIAS Y FRECUENCIA

Se ha implementado una estrategia mixta para optimizar el espacio de almacenamiento y la velocidad de ejecución:

- **Copia Completa (Full Backup):** Se realiza una vez a la semana (domingos a las 00:00). Contiene la totalidad de los datos.
- **Copia Incremental:** Se realiza diariamente de lunes a sábado. Solo guarda los cambios realizados desde el último respaldo (ya sea completo o incremental).
- **Justificación:** Esta combinación permite tener puntos de recuperación diarios consumiendo mucho menos espacio en disco que si hiciéramos copias completas cada día.

11.4.- UBICACIÓN DE LAS COPIAS

Para evitar que un fallo físico en el servidor destruya también los backups, se sigue una política de redundancia:

- **Copia Local:** Una copia permanece en el servidor en una partición aislada para restauraciones rápidas.
- **Copia Remota (Off-site):** Los respaldos se cifran y se suben automáticamente a un almacenamiento externo.
- **Aislamiento:** La ubicación remota es inaccesible desde la red pública, requiriendo llaves privadas para su gestión.

11.5.- PROCEDIMIENTO Y RESTAURACIÓN

Se ha definido un protocolo estándar de recuperación para asegurar la continuidad del negocio:

- **Preparación:** Despliegue de una nueva instancia Debian 13 con Docker instalado.
- **Configuración:** Restaurar docker-compose.yml, nginx.conf y archivos .env desde el backup y ejecutar docker compose up -d.
- **Importación de Datos:** Restauración de la última copia completa y aplicación sucesiva de las copias incrementales hasta la fecha del incidente (docker exec -i db-mysql mysql -u root -p Usuarios < backup.sql).
- **Validación:** Pruebas de integridad de datos y verificación de conectividad SSL antes de redirigir el tráfico del balanceador al nuevo nodo.

12.- ALTA DISPONIBILIDAD Y TOLERANCIA A FALLOS

12.1.- ANÁLISIS DE PUNTOS CRÍTICOS (SPOF)

Dada la arquitectura de nodo único para la base de datos, se identifica este componente como el principal Punto Único de Fallo.

- **Riesgo:** Una parada por mantenimiento, un fallo de hardware o una corrupción del sistema de archivos en el nodo de datos resultaría en la indisponibilidad total de la persistencia del sistema.
- **Mitigación:** Se compensa la falta de redundancia mediante un robusto sistema de copias de seguridad externas y una configuración de servidor optimizada para la estabilidad (restart: always en Docker Compose).

12.2.- MECANISMOS DE REDUNDANCIA Y RESILIENCIA

A pesar de la naturaleza monolítica de la base de datos, el sistema implementa resiliencia en otras capas:

- **Capa Web:** Se mantienen dos instancias del servidor web (backend1 y backend2) tras el balanceador, de modo que el fallo de un nodo de aplicación no afecte a la disponibilidad del servicio.
- **Redundancia de Datos (Pasiva):** Se mantiene una copia de los volcados de datos en un almacenamiento independiente. Esto permite levantar un servidor idéntico en un tiempo reducido.

12.3.- ESCENARIO DE FALLOS Y RESPUESTA DEL SISTEMA

Al no contar con un failover automático para la base de datos, el protocolo de respuesta se define de la siguiente manera:

Escenario de Fallo	Detección	Acción de Respuesta
Caída del servicio de BD	Monitorización de procesos / docker stats	Intento de reinicio automático (restart: always). Si persiste, alerta al administrador.
Corrupción de Disco	Alerta de E/S de hardware	Despliegue de un nuevo nodo y restauración de la última copia de seguridad íntegra.
Carga excesiva (CPU/RAM)	Alertas de umbral (>90%)	El balanceador puede limitar las conexiones para evitar el colapso total del nodo monolítico.
Caída de un backend PHP	Nginx health check pasivo	Nginx deja de enviar tráfico al backend caído. El otro backend asume toda la carga.

12.4.- RECUPERACIÓN ANTE DESASTRES

Al ser una arquitectura monolítica, la recuperación se centra en el Tiempo de Restauración (RTO).

- **Estrategia de Re-provisión:** En caso de desastre total del servidor de base de datos, se procede a la creación de una nueva instancia Debian 13 utilizando scripts de automatización y Docker Compose.
- **Restauración de Punto en el Tiempo:** Se importa el último volcado de datos disponible. El sistema volverá a estar operativo con los datos existentes hasta la última ejecución del backup (RPO).
- **Escalabilidad Futura:** El diseño actual contempla la migración a un modelo replicado o clúster de forma transparente, gracias a que el servidor web utiliza una cadena de conexión desacoplada mediante variables de entorno.

13.- SEGURIDAD

13.1.- GESTIÓN DE USUARIOS Y ACCESO

Se implementa un control de acceso basado en el Principio de Menor Privilegio (PoLP):

- **Acceso a nivel de OS:** Se deshabilita el acceso directo como usuario "root" vía SSH. Los administradores deben acceder mediante usuarios nominales y escalar privilegios mediante "sudo".
- **Autenticación SSH:** Se restringe el acceso mediante contraseña, permitiendo únicamente la autenticación a través de llaves SSH (RSA de 4096 bits / Ed25519).
- **Usuarios de Aplicación:** Los servicios (Web y DB) se ejecutan bajo usuarios de sistema dedicados y sin privilegios de shell, limitando el impacto en caso de una vulneración del servicio. En el entorno Windows Server, la gestión de altas y bajas de usuarios de Active Directory se realiza a través de la aplicación Python gestor_usuarios.py, que ofrece una interfaz gráfica (Tkinter) para ejecutar scripts PowerShell de forma controlada.
- **Passbolt:** Proporciona gestión de contraseñas corporativas con cifrado OpenPGP en el cliente, auditoría de accesos y permisos por equipos.

13.2.- POLÍTICAS DE CONTRASEÑAS Y SECRETOS

Para los servicios que requieren credenciales (Base de Datos, Paneles de administración):

- **Complejidad:** Uso obligatorio de contraseñas de alta entropía (mínimo 16 caracteres, incluyendo símbolos y alfanuméricos).
- **Cifrado:** Las contraseñas en la base de datos se almacenan utilizando algoritmos de hash seguros como Argon2 o BCrypt.
- **Gestión de Secretos:** Las credenciales de conexión entre el servidor web y la base de datos no se incluyen en el código fuente; se gestionan mediante variables de entorno en docker-compose.yml. En producción se recomienda usar Docker Secrets.

13.3.- ACTUALIZACIONES Y PARCHES DE SEGURIDAD

Se establece un ciclo de mantenimiento para garantizar la resiliencia frente a vulnerabilidades conocidas (CVE):

- **Actualizaciones Automáticas:** Configuración de "unattended-upgrades" para parches críticos de seguridad del sistema operativo Debian 13.
- **Imágenes Docker:** Se actualizan periódicamente usando docker compose pull y docker compose up -d.
- **Auditoría de Dependencias:** Uso de herramientas de escaneo para identificar y corregir librerías vulnerables en el código de la aplicación.

13.4.- PROTECCIÓN FRENTE A ATAQUES

Se han configurado defensas específicas para mitigar las amenazas más frecuentes en entornos web:

- **Firewall:** Uso de iptables con política FORWARD DROP y fail2ban para bloquear automáticamente direcciones IP que presenten comportamientos sospechosos (intentos fallidos de login, escaneo de puertos).
- **Protección contra Inyecciones:** Uso estricto de consultas preparadas (Prepared Statements / PDO) en la comunicación con la base de datos para anular ataques de SQL Injection.

- **Cabeceras de Seguridad:** Implementación de políticas como "Content Security Policy" (CSP), "X-Frame-Options" y "HSTS" en Nginx para proteger al usuario final contra ataques de XSS y Clickjacking.

13.5.- BUENAS PRÁCTICAS DE SEGURIDAD

Como cierre del esquema de seguridad, se siguen estándares de la industria:

- **Minimización de Superficie:** Se cierran todos los puertos que no sean estrictamente necesarios (solo 80, 443 en LAN 2 y 80, 443, 8080, 4443, 8081, 40443 internamente en LAN 1).
- **Logs de Auditoría:** Registro centralizado de eventos de seguridad para permitir el análisis forense tras cualquier incidente.
- **Desacoplamiento:** El servidor de base de datos no es accesible desde la red pública; sólo responde a peticiones provenientes del segmento de red interna (red-privada Docker).

14.- MONITORIZACIÓN Y MANTENIMIENTO

14.1.- HERRAMIENTA DE MONITORIZACIÓN

Para asegurar la visibilidad total sobre el estado de la infraestructura, se ha implementado un stack de monitorización compuesto por:

- **Monitorización de Contenedores:** docker stats para una visión rápida; cAdvisor integrado con Prometheus para métricas históricas de contenedores Docker.
- **Monitorización de Servicios:** Verificación de disponibilidad de procesos críticos (Nginx, MySQL) mediante chequeos externos e internos. El script certificados.sh supervisa automáticamente todos los certificados SSL del sistema y envía alertas por correo electrónico (vía msmtplib) cuando quedan menos de 30 días para la expiración.

14.2.- PARÁMETROS SUPERVISADOS

El sistema vigila de forma ininterrumpida los siguientes indicadores clave de rendimiento (KPIs):

- **Recursos de Hardware:** Consumo de CPU, uso de Memoria RAM y espacio en disco (vital para el nodo monolítico de base de datos).
- **Rendimiento de Red:** Ancho de banda entrante/saliente y latencia entre el balanceador de carga y los servidores backend.
- **Estado de Servicios:** Tiempo de actividad (uptime) de los contenedores Docker y el volumen de errores HTTP (clase 4xx y 5xx).
- **Base de Datos:** Número de conexiones activas, tiempos de ejecución de consultas y tamaño de los índices.
- **Certificados SSL:** Validez y fecha de expiración de los certificados autofirmados.

14.3.- GESTIÓN DE ALERTAS

Se ha configurado un sistema de notificaciones escalonado para garantizar una respuesta rápida ante incidencias:

- **Umbrales de Advertencia (Warning):** Se dispara una notificación (ej: vía Slack o Email) cuando el uso de recursos supera el 70% de su capacidad.
- **Alertas Críticas:** Si un servicio esencial se detiene o el uso de recursos supera el 90%, se activa una alerta de alta prioridad.
- **Silenciamiento programado:** Las alertas se desactivan automáticamente durante las ventanas de mantenimiento técnico para evitar falsos positivos.

14.4.- TAREAS DE MANTENIMIENTO PERIÓDICO

Para garantizar la longevidad y estabilidad del entorno, se ejecutan las siguientes rutinas:

- **Mantenimiento Preventivo:**
 - Semanal: Limpieza de archivos temporales y rotación de logs para liberar espacio en disco (docker system prune).
 - Mensual: Actualización de imágenes Docker (docker compose pull), revisión y aplicación de parches de seguridad del sistema operativo Debian 13, y prueba de restauración de backup.
 - Trimestral: Optimización de tablas de la base de datos (ANALYZE TABLE, OPTIMIZE TABLE) para mejorar el rendimiento de las consultas.

- **Auditoría de Seguridad:** Revisión periódica de los intentos de acceso fallidos y actualización de las reglas del firewall.

15.- PROCEDIMIENTOS OPERATIVOS

15.1.- ARRANQUE Y PARADAS DE SERVICIOS

Con Docker Compose, el orden de arranque está gestionado por la directiva `depends_on`. Para garantizar la integridad de los datos, el orden de manipulación de los servicios es crítico:

- **Secuencia de Arranque:** 1. Base de Datos (db-mysql) → 2. Servidores Web (backend1, backend2) → 3. Balanceador de Carga (nginx-balanceador).
- **Secuencia de Parada:** 1. Balanceador (para dejar de recibir tráfico) → 2. Servidores Web → 3. Base de Datos.
- **Comandos:** `docker compose up -d` (arranque) / `docker compose down` (parada). Uso de la directiva `restart: always` para inicio automático tras reinicio del host.

15.2.- ALTA Y BAJA DE UN SERVIDOR WEB

Gracias al balanceador de carga Nginx, el escalado o mantenimiento de la capa web es transparente:

- **Baja:** `docker stop backend1`. Nginx detecta el backend caído y deja de enviarle tráfico automáticamente. Una vez sin tráfico, se detiene el servicio.
- **Alta:** `docker start backend1`. Los health checks de Nginx validarán el nodo antes de enviarle usuarios reales.

15.3.- ACTUALIZACIÓN DEL SISTEMA

Se sigue el método de Actualización en Paralelo (Rolling Update) para la capa web y de Ventana de Mantenimiento para la base de datos:

- Actualizar imágenes: `docker compose pull`.
- Recrear contenedores de forma secuencial para la capa web: detener backend1, actualizarlo, arrancarlo; luego backend2.

- Para la base de datos (al ser única), se programa una ventana de baja actividad, se realiza un backup preventivo y se aplican los parches, asumiendo un breve periodo de inactividad técnica.

15.4.- RESTAURACIÓN DE BACKUPS

En caso de pérdida de datos en el servidor monolítico:

- Detener los servidores web para evitar escrituras inconsistentes: `docker stop backend1 backend2`.
- Limpiar la base de datos corrupta o levantar una instancia nueva.
- Ejecutar el comando de restauración: `docker exec -i db-mysql mysql -u root -p Usuarios < backup.sql`.
- Verificar la integridad de los datos y reiniciar los servicios web: `docker start backend1 backend2`.

16.- TROUBLESHOOTING

16.1.- PROBLEMAS HABITUALES

- **Error 502 Bad Gateway:** El balanceador no puede conectar con los servidores web. Verificar que backend1 y backend2 están corriendo (`docker ps`) y que la red-privada está activa.
- **Error 504 Gateway Timeout:** La aplicación o la base de datos están tardando demasiado en responder. Revisar conexiones activas en MySQL y carga del host.
- **Error de Conexión a la BD:** El servidor web no puede autenticarse o el nodo de base de datos ha alcanzado el límite de conexiones.
- **Certificado SSL expirado:** Verificar que el dominio aromaris.org resuelve correctamente y que el puerto 80 es accesible para que Let's Encrypt pueda renovar el certificado.
- **GLPI / Passbolt inaccesibles:** Verificar que los contenedores en LAN 1 están corriendo y que el proxy inverso (nginx-proxy) está funcionando en el puerto 443.

16.2.- DIAGNÓSTICOS BÁSICOS Y COMPROBACIONES

Ante una incidencia, se debe seguir la metodología top-down:

- Estado de contenedores: `docker ps -a` (ver todos los contenedores y su estado).
- Logs de un contenedor: `docker logs nginx-balanceador / docker logs db-mysql`.
- Conectividad entre contenedores: `docker exec -it backend1 ping db-mysql`.
- Recursos: `docker stats` (CPU y RAM de cada contenedor en tiempo real).
- Puertos activos en el host: `ss -tlnp`.

16.3.- LOGS RELEVANTES

Los registros son la fuente principal de verdad para el diagnóstico. Se deben consultar en este orden:

- **Balanceador:** `docker logs nginx-balanceador` → acceso y errores HTTP (códigos 4xx y 5xx).
- **Servidor Web (backends):** `docker logs backend1 / backend2` → errores de la aplicación PHP.
- **Base de Datos:** `docker logs db-mysql` → errores de MySQL, conexiones fallidas, consultas lentas.
- **Sistema (host):** `/var/log/syslog` o `/var/log/auth.log` para descartar bloqueos por seguridad o fallos de hardware.
- **Proxy Inverso LAN 1:** `docker logs <nombre-proxy-inverso>` → errores de enrutamiento hacia GLPI/Passbolt.